# Energy-Based Recurrent Transformers for Efficient Sequence Modeling

Kenju Tomita · kt7432@g.rit.edu
Rochester Institute of Technology
CSCI 635: Introduction to Machine Learning
Individual Project Proposal · October 9, 2026

## Abstract

Transformers model sequences effectively, but full attention requires a growing key/value cache during autoregressive inference. Recurrent models compress past observations into a fixed-size state, creating a tradeoff between memory efficiency and information retention. I will implement and train an energy-based recurrent Transformer from scratch, using quadratic energy coefficients as memory, and a hybrid that combines this recurrence with local attention. I will compare full attention, local attention, pure energy recurrence, and the hybrid on byte-level language modeling and controlled associative recall. All models will start from random weights. Experiments will match parameters and training data, repeat three seeds, and measure prediction loss, recall accuracy, memory, and runtime. A separate time-matched comparison will examine whether any prediction advantage survives implementation overhead. The goal is to identify when recurrent energy memory is useful and where it fails through a reproducible, semester-scale study.

## 1. Introduction and Research Objectives

Sequence modeling supports applications such as language prediction and processing continuous text streams. The Transformer introduced attention as a central mechanism for modeling relationships between tokens [1]. Full causal attention retains representations of the preceding sequence; its attention computation and inference cache become more expensive as sequences grow. Recurrent formulations offer another approach: update a bounded state instead of storing every token. Linear attention demonstrates a connection between Transformer-style computation and recurrent networks [2]. Energy-based architectures additionally provide a way to describe memory and retrieval through an energy function [3].

My proposed model uses a recurrent quadratic memory with learned writing, forgetting, and reading. The hybrid repeats two energy blocks followed by one causal local-attention block. Recurrence can retain compressed information across the stream, while local attention preserves detailed nearby interactions. The project asks: **Does this combination provide a better tradeoff between predictive quality, memory usage, and computation than comparable attention-only models?**

Existing exploratory results motivate, but do not settle, this question. A three-seed CPU study of approximately 145,000-parameter models reported 2.512 test bits per byte for the hybrid versus 2.578 for full attention and 2.577 for local attention under equal steady-state update-time budgets [5]. However, a historical 10.6-million-parameter comparison favored full attention over pure energy at equal training bytes [6]. These protocols differ and cannot establish a scaling trend. These studies are background evidence. For this project, I will write a separate PyTorch implementation and generate new results from randomly initialized models.

## 2. Methodology and Model

**Research hypotheses.** H1: The hybrid achieves lower held-out bits per byte than both attention controls at equal parameters and presented training bytes. H2: Energy recurrence and the hybrid maintain bounded carried inference state as prefix length increases. H3: The hybrid retains more information than local attention on queries whose supporting observations lie outside the local window. Each hypothesis may be rejected; reduced state alone does not establish better recall or faster execution.

**Energy recurrence.** For one attention head in transported slot coordinates, let $A_t$ represent accumulated precision above a positive prior $p_0$, and $S_t$ represent information stored in each slot. Learned projections produce keys $k_t$, values $v_t$, write strengths $w_t \geq 0$, and forgetting factors $f_t \in (0,1)$:

$$A_t = f_t \odot A_{t-1} + w_t k_t^2$$
$$S_t = f_t \odot S_{t-1} + w_t k_t v_t^T, \; P_t = p_0 + A_t$$

The memory energy is

$$E_t(M) = \tfrac{1}{2} \sum_j P_{t,j} ||M_j||^2 - \sum_j S_{t,j}^T M_j$$

Its unique minimum is $M^*_{t,j} = S_{t,j}/P_{t,j}$. Two learned query reads will be combined to produce the block output. Residual connections, normalization, and feedforward layers complete the model. Model weights are trained through backpropagation using next-byte cross-entropy. I will build embeddings, recurrence, attention, normalization, feedforward blocks, and the output head using PyTorch tensor operations and autograd. This is an independent implementation of the proposed memory formulation.

**Models and data.** The core comparison contains four families: full causal attention; local attention with a 64-byte window; pure energy recurrence; and the two-energy/one-local hybrid. I will use a 256-byte vocabulary and the enwik8 corpus, which contains Wikipedia text and markup [4]. The first 90 million bytes will form training, the next 5 million validation, and the final 5 million test. I will record the corpus checksum and sample windows within split boundaries. Test evaluation will use fixed, nonoverlapping windows with a declared state-reset policy.

The core grid uses approximately 145,000 parameters and three seeds per family: 12 main fits. A second size near 1.05 million parameters adds 12 fits if implementation and runtime permit. Feedforward widths will adjust parameter counts to within 0.5% at each size. All models will receive paired minibatch streams, a 512-byte training context, the same effective batch size, AdamW, and the same learning-rate search allowance. A separate calibration seed will select among three peak learning rates using validation only. Profiling will fix one common presented-byte budget before main training. Repeated and unique training bytes will be recorded separately. If resources cannot support the full grid, I will retain all families and seeds at the smaller size.

**Controlled recall and ablations.** Separate associative-recall sequences will contain randomly generated key/value associations, distractors, and a final query. Train, validation, and test episodes will use separate random streams. Delays of 32, 64, 128, and 256 bytes will expose the local window's limits. Accuracy will be measured only at answer positions. Pure energy and attention controls provide the principal architecture ablations; a fixed-forgetting variant of the hybrid will test whether learned forgetting helps, if time permits.

## 3. Evaluation, Feasibility, and Expected Contribution

**Evaluation.** The primary language metric is held-out bits per byte, computed from next-byte cross-entropy. I will report per-seed results, means, paired differences, training curves, and block-level uncertainty. Recall accuracy will measure memory directly. I will profile prefill and decoding at 512-, 2,048-, and 8,192-byte prefixes, documenting batch size, precision, warmup, and timing variability. Full attention will use a preallocated cache. Carried state, peak training memory, and parameter memory will be reported separately. Evaluation beyond the training context will be labeled context extrapolation.

Data-matched training and time-matched training answer different questions. At the smaller size, an additional three-seed comparison will give each family an equal end-to-end time allowance that includes setup and evaluation. Learning-rate and checkpoint selection will use validation only; final test results will not guide subsequent changes. Numerical checks will cover causality, chunked versus token-by-token execution, nonfinite values, and checkpoint restart behavior.

**Feasibility.** I will first implement a readable serial recurrence and a minimal attention baseline, then build the data loader, trainer, checkpoints, and evaluator. Chunked optimization will follow numerical parity checks. I will request available local or RIT GPU access and cap new GPU experiments at 12 hours: two for correctness/profiling, eight for training, and two for evaluation. Profiling will reserve a 20% timing margin when choosing the common training budget. If GPU access is limited, the replicated 145,000-parameter study remains the minimum deliverable. The study will characterize small-model trends without extrapolating to foundation-model scale.

**Individual work plan.** I will implement and train every model without importing existing project modules or weights, conduct analysis and writing, credit external resources, and arrange peer proofreading.

| Dates | Milestone and output |
|---|---|
| October 9 | Submit proposal; define hypotheses, controls, and measurements. |
| October 10–16 | Implement embeddings and serial recurrence; verify fixtures and present the proposal on October 16. |
| October 17–November 1 | Complete models, data pipeline, trainer, and checks; freeze the protocol and calibrate learning rates. |
| November 2–15 | Train the replicated grid from random weights; run recall tests and add the second size if feasible. |
| November 16–29 | Run time-matched confirmation and inference profiling; freeze selections and evaluate test data. |
| November 30–December 8 | Analyze results, create figures, obtain peer feedback, and finish the report and reproducibility README. |
| December 9 | Submit final report and code/artifacts as required. |
| December 10–13 | Rehearse and give the final presentation on December 13. |

**Expected contribution.** Deliverables include independently implemented code, experiment specifications, checkpoints/logs, quality-versus-cost figures, and a recall-failure analysis. Both favorable and unfavorable results will help identify where recurrent energy memory is useful.

## References

1. Vaswani, A., et al. “Attention Is All You Need.” *NeurIPS*, 2017. Conference proceedings.

2. Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. “Transformers Are RNNs: Fast Autoregressive Transformers with Linear Attention.” *ICML*, PMLR 119:5156–5165, 2020. Conference proceedings.

3. Hoover, B., et al. “Energy Transformer.” *NeurIPS*, 2023. Conference proceedings.

4. Mahoney, M. “About the Test Data,” Large Text Compression Benchmark. Dataset source and checksums.

5. EnergyTransformer project. “Compute-Controlled Confirmation.” Existing local exploratory report, September 2026. Local report (included in project repository).

6. EnergyTransformer project. “Reconciled Evidence and Repair Results.” Existing local historical-results audit, September 2026. Local report (included in project repository).